

Aspire — System Architecture and Latency Performance Report

Real-time accessibility narration for slide presentations

Azizul Haque · Stevens Institute of Technology

Report date: 26 July 2026

1. Executive Summary

Aspire captures a presenter's screen and system audio, automatically detects slide changes, generates spoken descriptions of each new slide with a vision-language model, mixes those descriptions into the presenter's audio at natural pauses, and streams the result to blind and low-vision (BLV) listeners on their phones.

The system worked correctly but delivered audio **25–30 seconds behind live**, which made it unusable as a live-listening aid. This report documents the measurement campaign that located the delay, the changes made, and the result.

Headline result: end-to-end listener delay reduced from 25–30 s to 4.0 s (median), with audio dropouts eliminated entirely.

Table 1. Summary of measured outcomes.

Measure	Before	After
End-to-end listener delay (median)	25–30 s	4.0 s
End-to-end listener delay (95th percentile)	~30 s	8.7 s
Session under 10 s behind live	~0 %	97 %
Session under 5 s behind live	~0 %	66 %
Audio dropouts (hard cuts in playback)	67 per session	0
Vision-model API calls per session	184	39
Screen area analysed	34 %	92 %
Slides described more than once	Yes	None

A key finding is that the delay was **not** caused by the cloud vision model, which was the initial hypothesis. The model accounted for approximately 6 % of the delay. The dominant causes were an audio backlog that could never drain and a conservative streaming buffer.

2. System Architecture

Aspire is a pipeline of seven stages running on a single Apple Silicon Mac.

Table 2. Pipeline stages and the tools implementing them.

#	Stage	Function	Tool / Model
1	Screen capture	1280×720 video at 5 fps	ffmpeg (avfoundation)
2	Audio capture	System audio, 24 kHz → 48 kHz stereo	ScreenCaptureKit via a Swift CLI (SystemAudioDump)
3	Slide detection	Perceptual hash comparison to detect visual change	ImageHash 4.3.1 (pHash)
4	Text extraction	On-device OCR, used as ground truth for titles and for de-duplication	Apple Vision framework (pyobjc 12.1)
5	Description	One short spoken-form sentence per slide	Claude Haiku 4.5 (anthropic 0.97.0)
6	Speech synthesis	Description rendered to 48 kHz audio	Kokoro ONNX 0.5.0, voice af_heart at 1.5×
7	Mixing and delivery	50 fps mixer inserts descriptions at natural pauses; streams to listeners	NumPy mixer; aiortc 1.14.0 (WebRTC), HLS fallback; FastAPI 0.112.0 / uvicorn

Supporting components: `faster-whisper` ("tiny", int8, CPU) for filler-word detection, and a Cloudflare quick tunnel for listeners outside the local network.

2.1 Core design concept: delay-buffered editing

Aspire does not play descriptions over the presenter. It holds the presenter's audio in a buffer, waits for a natural pause (a silence of ≥ 0.30 s), and inserts the description there. Because every inserted description adds its own duration to the buffer, the system must continuously *reclaim* time to prevent the listener from falling progressively further behind. How that time is reclaimed is the central performance question of this report.

3. Measurement Methodology

All figures in this report are measured, not estimated. Instrumentation already present in the system was supplemented with two 1 Hz pollers.

Table 3. Telemetry sources.

Source	Content
<code>aspire-diag-events.jsonl</code>	Structured event log, one JSON object per event, ~300 events/second across 25 event types (capture reads, queue operations, mixer ticks, detection, model calls, delivery)
<code>backend.log</code>	Text log including a per-slide latency decomposition line
<code>/webrtc/status</code> (1 Hz)	Buffer depth, catch-up rate, cumulative counters
<code>/diagnostics/snapshot</code> (1 Hz)	Per-listener playback position — the ground truth for listener delay

Six full sessions were recorded (approximately 3.5 hours of talk audio): four baseline sessions on the original system, and two sessions during the optimisation. Each session produced 550,000–740,000 diagnostic events.

3.1 Defining the delay correctly

An early analytical error is worth recording, because it changes the conclusion. The per-listener metric `behind_live_ms` measures the listener's position relative to the **mixer's output**, not relative to the **live presenter**. The mixer's output is itself delayed by the server-side audio backlog. The true end-to-end delay is therefore:

$$\text{end-to-end delay} = \text{server audio backlog} + \text{listener delivery lag}$$

Reporting only `behind_live_ms` would have understated the true delay by a factor of three.

4. Diagnosis of the Original Delay

4.1 Two independent delays

Measured across the four baseline sessions:

Table 4. Decomposition of the original 25–30 s delay.

Component	Measured	Share
Server-side audio backlog	14.8 – 22.0 s (median), peaking at the 30 s cap	~65–75 %
Streaming delivery buffer (HLS)	5 – 11 s	~25–35 %
Total	25 – 30 s	100 %

4.2 Primary cause: the backlog could never drain

The decisive evidence is that the mixer was in its CATCHUP state — meaning it was actively trying to catch up — for **75 %, 87 %, 88 % and 86 %** of the four sessions respectively. It was almost permanently behind and never recovering.

The reason is structural. Each spoken description holds back the presenter's audio for 5–8 seconds, and that speech accumulates in the buffer. The original system had only two mechanisms to reclaim that time: trimming silences longer than 1 second, and removing filler words ("um", "uh") detected by Whisper.

Both only work when the speaker pauses. Against a continuous speaker there is nothing to remove. Measured over one 35-minute session, these mechanisms together reclaimed just **4.4 seconds** (2.7 s of filler, 1.7 s of silence) — against a debt of roughly 7 seconds *per description*. The buffer therefore grew until it struck its 30-second ceiling, at which point the system discarded half the buffer, causing audio loss.

4.3 Secondary cause: conservative streaming buffer

Listeners received audio via HLS, which transmits audio as 2-second file segments. The player was configured with a 5-second pre-buffer and a 6-second live-edge offset, adding a further 5–11 seconds. This configuration is safe against network interruption but expensive in latency.

4.4 The cloud model was not the bottleneck

The initial hypothesis was that the Anthropic API was responsible, and that replacing it with a locally-hosted model would reduce the delay. Measurement of 62 described slides did not support this:

Table 5. Per-stage processing time, 62 described slides.

Stage	Median
Vision model call (Claude Haiku 4.5, cloud)	1.81 s
Speech synthesis (Kokoro, local)	2.50 s
Waiting for a natural pause	1.80 s

The cloud model accounted for approximately **6 %** of the total delay, and was *faster* than the locally-run speech synthesis. Published results for a comparable system support this: WorldScribe (Chang et al., UIST 2024) reports ~3 s for the compact local vision model Moondream running on two RTX 4090 GPUs, against 1.81 s measured here for a cloud call on a laptop. **Switching to a local model would have increased latency, not reduced it.** The relevant lesson from that work was not the choice of model but its pipeline structure: begin all description work the moment a keyframe arrives.

5. Changes Implemented

Table 6. Methods and tools, before and after.

Area	Before	After
Time reclamation	Silence trimming and filler removal only — effective only during pauses	Overlap-add time-stretch : buffered speech plays up to 1.25× faster, pitch preserved, draining the backlog continuously
Listener transport	HLS with 5 s pre-buffer and 6 s live offset	WebRTC by default (aiortc), ~60 ms transport lag; HLS retained as fallback and retuned to 1 s / 2 s
Delivery pacing	Fixed 20 ms sleep per frame	Absolute wall-clock deadline pacing, with a gentle single-frame correction
Slide region	Automatic region detection	Full-frame analysis (92 % of screen); automatic detection available via environment variable
De-duplication	Perceptual hash + OCR text	Added description-level de-duplication , including short-title and same-title cases
Description wait cap	8.0 s	2.5 s
Slide stability window	5.0 s	3.0 s

5.1 Time-stretch catch-up (principal change)

To reclaim time without deleting speech, the mixer consumes **two** buffered 20 ms frames and emits **one** frame that cross-fades between them. This removes 20 ms of timeline while preserving pitch, and the cross-fade prevents the click that simply discarding a frame would produce.

The rate is proportional to how far behind the listener is: 1.00× at or below a 2-second backlog (natural speech, no processing), ramping linearly to 1.25× at a 15-second backlog. The system therefore speeds up only when it needs to, and returns to unmodified audio once caught up.

Implemented in approximately ten lines of NumPy with no additional dependency and no subprocess, so it introduces no risk to the real-time 50 fps mixing loop. In the final session it reclaimed **55.4 seconds** of listener delay.

5.2 Elimination of audio dropouts

Listeners reported the audio "cutting" mid-description. The cause was located in the logs: **67 events per session — one roughly every 31 seconds — each skipping exactly 201 frames (4.02 seconds) of audio.**

The WebRTC delivery track paced itself with a fixed 20 ms sleep. Because `asyncio.sleep()` always overshoots slightly, the listener drifted roughly 150 ms/second behind the source until crossing a 4-second threshold, at which point the system resynchronised by jumping forward — discarding four seconds of audio mid-sentence.

Pacing was changed to an absolute wall-clock deadline, which eliminates drift, plus a gentle single-frame (20 ms, inaudible) correction if a small gap appears. The count of these events in the verification session was **zero**.

It is worth noting that the initial hypothesis — that filler and pause removal were cutting the audio too aggressively — was disproved by measurement: those mechanisms removed only 4.4 seconds across an entire 35-minute session.

5.3 Description quality

Two defects were corrected:

- **Partial screen analysis.** Automatic slide-region detection was cropping to a median of 34 % of the screen (as little as 4 %) and moved 133 times in a single session. An unstable crop also makes the *same* slide appear different on each sample, defeating de-duplication. Analysis now uses 92 % of the frame, trimming only the menu bar and dock, whose changing clock would otherwise defeat de-duplication.
- **Slides described twice.** A slide built up in stages (title first, then body) was described once as "Historically." and again as "Historically. FT All Share shows 4.5 %...". De-duplication now additionally compares the generated descriptions, handling both short titles and the case where two differently worded summaries share a title. In the verification session, no slide was described more than once.

6. Results

Measured over full sessions with a real listener connected.

Table 7. End-to-end listener delay.

Metric	Baseline (HLS)	Intermediate	Final
Median	25–30 s	6.5 s	4.0 s
95th percentile	~30 s	15.2 s	8.7 s
Under 5 s	~0 %	34 %	66 %
Under 10 s	~0 %	75 %	97 %
Audio dropouts	present	67	0

Table 8. Component detail.

Component	Baseline	Final
Server audio backlog (median)	14.8 – 22.0 s	3.9 s
Server audio backlog (95th percentile)	23.9 – 29.0 s	8.7 s
Listener delivery lag (median)	5 – 11 s	0.0 s
Listener delivery lag (maximum)	—	0.1 s
Mixer in CATCHUP state	75 – 88 %	intermittent

Table 9. Slide-to-speech latency.

Stage	Baseline	Final
Total (commit → audible)	7.39 s	5.17 s
— vision model	1.81 s	1.68 s
— speech synthesis	2.50 s	0.91 s
— waiting for a natural pause	1.80 s	2.51 s
Slide stability window (before commit)	5.0 s	3.0 s
Slide change → speech	12.4 s	8.2 s

Table 10. Processing efficiency.

Metric	Before	After
Vision-model API calls per session	184	39 (–79 %)
Calls discarded as no-content	148 (80 %)	21 (53 %)
Slides actually described	25	17

7. Why the System No Longer Lags

Three independent facts explain the result.

1. **The backlog now drains continuously rather than only during pauses.** Time reclamation is no longer conditional on the speaker being silent. At a $1.25\times$ catch-up rate, one minute of playback recovers fifteen seconds of delay, which comfortably exceeds the ~ 7 seconds of debt each description creates. The buffer settles at roughly 2–4 seconds instead of saturating at its 30-second ceiling.
2. **The delivery buffer was removed rather than merely reduced.** WebRTC transmits 20 ms packets over a live connection, with no file segmentation, no playlist, and no pre-buffer. Measured listener delivery lag is 0.0 s (median) and 0.1 s (maximum), against 5–11 s for HLS.
3. **Playback no longer drifts.** With deadline-based pacing, the listener stays locked to the source, so the 4-second corrective jumps that produced audible cuts no longer occur.

8. Limitations and Future Work

- **Design tension.** The system's contribution is *delay-buffered editing* — inserting descriptions into reclaimed time. At a 4-second delay there is less buffer to reclaim from, so descriptions are more likely to overlap the speaker than to fall into a natural gap. Time-stretch mitigates this but does not remove the tension; approximately 5–10 seconds appears to be a reasonable operating point that preserves the concept.
- **Residual delay is dominated by the backlog** (median 3.9 s). Further reduction would come from a higher catch-up rate or shorter descriptions, both single parameter changes.
- **Speculative generation is not yet implemented.** Beginning description work when a slide candidate first appears, rather than after the stability window, would hide a further 3–5 seconds. This is the structural idea drawn from WorldScribe.
- **Slide images are not retained**, so description accuracy must be rated against the source video rather than stored frames.
- **Reported figures come from six sessions** of lecture-style content on one hardware configuration. Denser slide sequences would increase the equilibrium backlog.

Appendix A — Key Configuration

Parameter	Value	Purpose
CATCHUP_TARGET_MS	2000	Backlog below which playback is unmodified
CATCHUP_RAMP_MS	15000	Backlog at which maximum catch-up rate is reached
CATCHUP_MAX_RATE	1.25	Maximum playback rate during catch-up
STABILITY_WINDOW_S	3.0	Hold before describing, allowing slide build-ups to finish
TTS_PAUSE_TRIGGER_S	0.30	Silence qualifying as a natural pause
TTS_HOLD_MAX_S	2.5	Maximum wait for a pause before speaking anyway
PEER_GENTLE_CATCHUP_FRAMES	15	300 ms drift before a single-frame correction
SILENCE_KEEP_MS	1000	Silence preserved before trimming begins
Frame size / rate	20 ms / 50 fps	Mixer timebase

Appendix B — Data Availability

Six instrumented sessions are archived, each containing the complete diagnostic event log, backend text log, both 1 Hz telemetry time-series, and a per-slide CSV of spoken descriptions with timestamps.